🏗️ Documentação Técnica - Arquitetura e Implementação

Índice

- Arquitetura Geral
- Estrutura de Pastas
- Padrões de Código
- Service Worker e PWA
- Firebase Integration
  - Autenticação e Controle de Acesso (Roles)
- Painel Administrativo
- Self-Edit (Meu Perfil)
- Push Notifications (FCM)
- React Query Strategy
- Tema e Estilização

Arquitetura Geral

O projeto segue uma arquitetura em camadas com separação clara de responsabilidades:

Camadas

```
┌──────────────────────────────────────┐
│     Presentation Layer (UI)          │  ← Components, Pages
├──────────────────────────────────────┤
│     Business Logic Layer             │  ← Hooks, Providers
├──────────────────────────────────────┤
│     Data Access Layer                │  ← Services, Queries
├──────────────────────────────────────┤
│     External Services                │  ← Firebase, APIs
└──────────────────────────────────────┘
```

Princípios Aplicados

- **Clean Architecture**: Separação entre UI, lógica e dados
- **SOLID**: Single Responsibility, Open/Closed, Dependency Inversion
- **DRY**: Componentes e hooks reutilizáveis
- **Composition over Inheritance**: Hooks customizados e HOCs

---

Estrutura de Pastas

```
lulu/
├── app/                          # Next.js App Router
│   ├── (routes)/                 # Rotas agrupadas
│   │   ├── dashboard/
│   │   ├── galeria/
│   │   ├── historico/
│   │   ├── auditoria/
│   │   └── participantes/
│   ├── api/                      # API Routes (se necessário)
│   ├── offline/                  # PWA offline page
│   ├── sw.ts                     # Service Worker (Serwist)
│   ├── layout.tsx                # Root layout
│   ├── page.tsx                  # Home page
│   ├── icon.png                  # App icon (metadata)
│   └── globals.css               # Global styles
│
├── components/                   # Componentes reutilizáveis
│   ├── ui/                       # Primitivos Radix UI estilizados
│   │   ├── button.tsx
│   │   ├── input.tsx
│   │   ├── dialog.tsx
│   │   ├── form.tsx
│   │   └── ... (20+ componentes)
│   │
│   ├── galeria/                  # Feature: Galeria
│   │   ├── galeria-fotos.tsx
│   │   ├── photo-card.tsx
│   │   ├── comment-section.tsx
│   │   ├── like-button.tsx
│   │   └── upload-form.tsx
│   │
│   ├── lulus/                    # Feature: Participantes
│   │   ├── form-edit-data/
│   │   │   ├── person-form.tsx   # Formulário (mode: admin | self-edit)
│   │   │   ├── gift-profile-form.tsx # Perfil de presente (wish list, tamanhos)
│   │   │   └── validation.ts     # Schemas Zod (person, gift, address)
│   │   ├── person-list.tsx
│   │   └── types.ts              # Person, Role, WishListItem, Address, ShirtSize
│   │
│   ├── vaquinha-history/         # Feature: Histórico
│   ├── audit/                    # Feature: Auditoria
│   ├── data-table/               # Tabela genérica (TanStack)
│   ├── layout/                   # Layout components
│   │   ├── navbar.tsx            # Nav com filtragem por role
│   │   ├── footer.tsx            # Footer com filtragem por role
│   │   └── pwa-update-manager.tsx
│   └── modules/                  # Módulos específicos
│       ├── admin/                # Painel administrativo
│       │   ├── admin-participant-form.tsx  # Cadastro de novas lulus
│       │   └── role-manager.tsx           # Gerenciamento de roles
│       └── notifications/        # Push Notifications
│           ├── notification-opt-in.tsx     # Opt-in UI
│           └── fcm-foreground-handler.tsx  # Handler foreground messages
```

```
│
├── services/                     # Business Logic + Firebase
│   ├── firebase.ts               # Firebase config & exports
│   ├── fcm.ts                    # Push notifications (FCM client)
│   ├── galeriaComments.ts        # CRUD Comentários
│   ├── galeriaLikes.ts           # CRUD Likes
│   ├── vaquinhaHistory.ts        # CRUD Histórico
│   ├── participants-server.ts    # Server-side participants
│   ├── participants-admin.ts     # CRUD Admin (cadastro, roles, exclusão)
│   │
│   ├── queries/                  # React Query hooks
│   │   ├── fetchParticipants.ts
│   │   ├── updateParticipant.ts  # Mutation com mode admin | self-edit
│   │   ├── adminParticipants.ts  # Hooks admin (create, delete, updateRole)
│   │   ├── useGalleryQueries.ts
│   │   └── useHistoryQueries.ts
│   │
│   └── audit/                    # Auditoria services
│
├── hooks/                        # Custom React Hooks
│   ├── use-disclosure.ts         # Modal state management
│   ├── user-verify.ts            # Auth + role + participantId
│   ├── use-auto-link-account.ts  # Auto-link Auth UID ↔ participant
│   ├── useUploadPhoto.ts         # Photo upload logic
│   └── usePwaUpdate.ts           # PWA update notifications
│
├── providers/                    # Context Providers
│   ├── react-query-provider.tsx  # TanStack Query config
│   ├── theme-provider.tsx        # next-themes wrapper
│   └── device-provider.tsx       # React device detect
│
├── lib/                          # Utilities & Helpers
│   ├── utils.ts                  # cn(), formatters
│   ├── crypto.ts                 # Encryption helpers
│   ├── auth-guard.ts             # assertAdmin(), assertOwnerOrAdmin()
│   ├── nav-config.ts             # Itens de navegação com requiredRole
│   └── i18n/                     # Internacionalização
│
└── public/                       # Static assets
    ├── manifest.webmanifest      # PWA manifest
    ├── sw.js                     # Compiled service worker
    ├── icons/                    # PWA icons (192, 512, maskable)
    ├── animation/                # Lottie animations
    └── fotos/                    # User uploaded photos
```

Padrões de Código

1. Componentes React

Client Components

Marcados com `'use client'` quando usam:

- Hooks (`useState`, `useEffect`, custom hooks)
- Event handlers
- Browser APIs
- Context consumers que não são server-safe

```
'use client';

import { useState } from 'react';
import { useUserVerification } from '@/hooks/user-verify';

export const GaleriaFotos = () => {
  const [filter, setFilter] = useState('');
  const { user, isLoading } = useUserVerification();

  // Component logic...
};
```

Server Components (Default)

Usados para:

- Data fetching inicial
- SEO metadata
- Componentes estáticos sem interatividade

```
// No 'use client' - é Server Component por padrão
import { Metadata } from 'next';

export const metadata: Metadata = {
  title: 'Dashboard',
  description: 'Dashboard de métricas',
};

export default async function DashboardPage() {
  const data = await fetchDashboardData();
  return <DashboardClient data={data} />;
}
```

2. Formulários (React Hook Form + Zod)

Padrão consistente em todo o projeto:

```
// 1. Schema de validação
const participantSchema = z.object({
  fullName: z.string().min(1, 'Nome obrigatório'),
```

```
  email: z.string().email('Email inválido').optional(),
  phone: z.string().optional(),
  instagram: z.string().optional(),
  pixKey: z.string().optional(),
});

type ParticipantForm = z.infer<typeof participantSchema>;

// 2. Hook form com resolver
const form = useForm<ParticipantForm>({
  resolver: zodResolver(participantSchema),
  defaultValues: initialData,
});

// 3. Mutation com React Query
const updateMutation = useUpdateParticipantData();

const onSubmit = (data: ParticipantForm) => {
  updateMutation.mutate(data, {
    onSuccess: () => {
      toast.success('Atualizado com sucesso!');
      form.reset();
    },
    onError: (error) => {
      toast.error('Erro ao atualizar');
    },
  });
};

// 4. Form component com UI components
<Form {...form}>
  <form onSubmit={form.handleSubmit(onSubmit)}>
    <FormField
      control={form.control}
      name="fullName"
      render={({ field }) => (
        <FormItem>
          <FormLabel>Nome Completo</FormLabel>
          <FormControl>
            <Input {...field} />
          </FormControl>
          <FormMessage />
        </FormItem>
      )}
    />
  </form>
</Form>
```

3. Estilo de Código

```
// ☑ Bom: Arrow functions, const, tipos explícitos
export const MyComponent = ({ name }: { name: string }) => {
  const handleClick = () => {
    console.log(name);
  };

  return <button onClick={handleClick}>{name}</button>;
};

// ❌ Evitar: function declarations
export function MyComponent({ name }: { name: string }) {
  function handleClick() {
    console.log(name);
  }
  return <button onClick={handleClick}>{name}</button>;
}
```

Service Worker e PWA

Implementação com Serwist

app/sw.ts - Service Worker

```
/// <reference lib="webworker" />
import { defaultCache } from '@serwist/next/worker';
import type { PrecacheEntry, SerwistGlobalConfig } from 'serwist';
import { Serwist } from 'serwist';

declare global {
  interface WorkerGlobalScope extends SerwistGlobalConfig {
    __SW_MANIFEST: (PrecacheEntry | string)[] | undefined;
  }
}

const swSelf = globalThis as unknown as WorkerGlobalScope;

const serwist = new Serwist({
  precacheEntries: swSelf.__SW_MANIFEST,
  skipWaiting: true, // Ativa nova versão imediatamente
  clientsClaim: true, // Assume controle dos clients existentes
  navigationPreload: true, // Preload durante navigation requests
  runtimeCaching: defaultCache, // Estratégias de cache padrão
  fallbacks: {
    entries: [
      {
        url: '/offline',
        matcher({ request }) {
          return request.destination === 'document';
        },
```

```
      },
    ],
  },
});

serwist.addEventListeners();
```

next.config.ts - Integração Serwist

```
import withSerwistInit from '@serwist/next';

const withSerwist = withSerwistInit({
  swSrc: 'app/sw.ts',
  swDest: 'public/sw.js',
  disable: process.env.NODE_ENV !== 'production',
  cacheOnNavigation: true,
  reloadOnOnline: true,
  additionalPrecacheEntries: [{ url: '/offline', revision }],
  exclude: [/.map$/, /^manifest.*\.js$/],
  globPublicPatterns: ['**/*.{js,css,html,ico,png,jpg,jpeg,svg,woff2}'],
});

export default withSerwist(nextConfig);
```

Estratégias de Cache

1. **Precache**: Assets críticos (JS, CSS, fonts)
2. **Network First**: Dados Firebase (sempre tenta rede primeiro)
3. **Cache First**: Imagens e assets estáticos
4. **Stale While Revalidate**: Navegação de página

PWA Update Manager

Notifica usuários sobre atualizações disponíveis:

```
export const PwaUpdateManager = () => {
  const { hasUpdate, updateServiceWorker } = usePwaUpdate();

  if (!hasUpdate) return null;

  return (
    <div className="fixed bottom-4 right-4 z-50">
      <Button onClick={updateServiceWorker}>
        Nova versão disponível! Atualizar
      </Button>
    </div>
  );
};
```

Firebase Integration

Configuração

```
// services/firebase.ts
import { initializeApp } from 'firebase/app';
import { getAuth } from 'firebase/auth';
import { getFirestore } from 'firebase/firestore';
import { getStorage } from 'firebase/storage';

const firebaseConfig = {
  apiKey: process.env.NEXT_PUBLIC_FIREBASE_API_KEY,
  authDomain: process.env.NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN,
  projectId: process.env.NEXT_PUBLIC_FIREBASE_PROJECT_ID,
  storageBucket: process.env.NEXT_PUBLIC_FIREBASE_STORAGE_BUCKET,
  messagingSenderId: process.env.NEXT_PUBLIC_FIREBASE_MESSAGING_SENDER_ID,
  appId: process.env.NEXT_PUBLIC_FIREBASE_APP_ID,
};

const app = initializeApp(firebaseConfig);

export const auth = getAuth(app);
export const db = getFirestore(app);
export const storage = getStorage(app);
```

Real-time Listeners Pattern

```
// services/galeriaLikes.ts
import {
  doc,
  onSnapshot,
  updateDoc,
  arrayUnion,
  arrayRemove,
} from 'firebase/firestore';
import { db } from './firebase';

export const subscribeToLikes = (
  photoId: string,
  callback: (users: string[]) => void
) => {
  const docRef = doc(db, 'galeria-likes', photoId);

  return onSnapshot(docRef, (snapshot) => {
    const data = snapshot.data();
    callback(data?.users || []);
  });
```

```
};

export const likePhoto = async (photoId: string, userId: string) => {
  const docRef = doc(db, 'galeria-likes', photoId);
  await updateDoc(docRef, {
    users: arrayUnion(userId),
  });
};

export const unlikePhoto = async (photoId: string, userId: string) => {
  const docRef = doc(db, 'galeria-likes', photoId);
  await updateDoc(docRef, {
    users: arrayRemove(userId),
  });
};
```

Upload de Fotos

```
// hooks/useUploadPhoto.ts
import { ref, uploadBytesResumable, getDownloadURL } from 'firebase/storage';
import { storage } from '@/services/firebase';

export const useUploadPhoto = () => {
  const [progress, setProgress] = useState(0);
  const [url, setUrl] = useState<string | null>(null);

  const upload = (file: File) => {
    const storageRef = ref(storage, `galeria/${Date.now()}_${file.name}`);
    const uploadTask = uploadBytesResumable(storageRef, file);

    uploadTask.on(
      'state_changed',
      (snapshot) => {
        const progress =
          (snapshot.bytesTransferred / snapshot.totalBytes) * 100;
        setProgress(progress);
      },
      (error) => console.error(error),
      async () => {
        const downloadURL = await getDownloadURL(uploadTask.snapshot.ref);
        setUrl(downloadURL);
      }
    );
  };

  return { upload, progress, url };
};
```

Autenticação e Controle de Acesso (Roles)

O sistema suporta **três papéis (roles)**: `admin`, `lulu` e `visitante`.

- **admin**: Custom Claim no JWT do Firebase + campo `role` no Firestore
- **lulu**: Participante vinculada, pode editar o próprio perfil
- **visitante**: Padrão para qualquer usuário autenticado sem vínculo

A identidade é validada em três camadas:

```
┌──────────────────────────────────────────────────────────┐
│  1. UI          → nav/botões filtrados por role          │
│  2. Serviço     → assertAdmin() / assertOwnerOrAdmin()   │
│  3. Firestore/Storage Rules → servidor                   │
└──────────────────────────────────────────────────────────┘
```

Fluxo de autenticação

```
Login → onAuthStateChanged →
  getIdTokenResult(user) → setIsAdmin(!!claims.admin) →
  query('participants', where('authEmail', '==', email)) →
  setRole(data.role) + setParticipantId(doc.id)
```

O hook `useUserVerification()` retorna:

```
{
  user, isLogged, isAdmin, isLulu, role, participantId, isLoading, handleLogout;
}
```

Auto-Link de Conta

O hook `useAutoLinkAccount(user)` vincula automaticamente o UID do Firebase Auth ao documento do participante quando `authEmail` bate com o e-mail do usuário logado. Usa `useRef` para evitar execuções duplicadas.

Guard de serviço

```
// lib/auth-guard.ts
export const assertAdmin = async (): Promise<void> => {
  const currentUser = auth.currentUser;
  if (!currentUser) throw new Error('Usuário não autenticado');
  const tokenResult = await getIdTokenResult(currentUser, true);
  if (!tokenResult.claims.admin)
    throw new Error('Acesso restrito a administradores');
};
```

```
export const assertOwnerOrAdmin = async (
  participantId: string
): Promise<void> => {
  // Permite se admin; caso contrário, verifica se data.uid === currentUser.uid
};
```

Navegação Filtrada por Role

`lib/nav-config.ts` define `requiredRole?: 'admin' | 'lulu'` em cada `NavItem`. Navbar e Footer filtram itens de acordo com `isAdmin` e `role` do usuário:

- `/admin` → visível apenas para admins
- `/meu-perfil` → visível para lulus (e admins)
- Demais rotas → visíveis para todos

Definir o primeiro admin (CLI)

```
# Conceder admin (por e-mail ou UID)
npx ts-node scripts/set-admin.ts oliver.sueli@gmail.com
npx ts-node scripts/set-admin.ts 3PAs2kahMjde4eqV6TObJ8sXXvz2

# Revogar admin
npx ts-node scripts/set-admin.ts oliver.sueli@gmail.com --revoke

# Em ambientes sem serviceAccountKey.json (CI/CD)
FIREBASE_SERVICE_ACCOUNT_KEY='...' npx ts-node scripts/set-admin.ts <email-ou-uid>
```

> Após rodar o script, faça **logout e login** na aplicação. O JWT é cacheado e só atualiza no próximo login.

Conceder ou revogar admin via API (após ter um admin)

```
# Conceder
POST /api/admin/set-claim
Authorization: Bearer <id-token-do-admin>
Content-Type: application/json
{ "targetUid": "<uid-do-usuario>" }

# Revogar
POST /api/admin/set-claim
Authorization: Bearer <id-token-do-admin>
Content-Type: application/json
{ "targetUid": "<uid-do-usuario>", "admin": false }
```

O endpoint verifica o token do caller, confirma `admin: true` no claim, valida que o `targetUid` existe e faz merge com claims existentes antes de aplicar.

Permissões por coleção (Firestore)

Coleção	Leitura	Criar/Excluir	Atualizar
`participants`	Público	Admin	Admin OU owner (sem alterar `role`, `uid`, `authEmail`)
`participants/audit`	Público	Autenticado	Autenticado
`vaquinha-history`	Público	Admin	Admin
`galeria-likes`	Público	Autenticado	Autenticado
`galeria-comments`	Público	Autenticado	Autenticado

Funções helper nas Firestore Rules:

- `isOwner(participantId)` → `resource.data.uid == request.auth.uid`
- `isNotChangingProtectedFields()` → impede alteração de `role`, `uid`, `authEmail`

```
# Deploy das regras
firebase deploy --only firestore:rules,storage
```

Variável de ambiente em produção

Em produção, **não use** o arquivo `serviceAccountKey.json`. Configure:

```
FIREBASE_SERVICE_ACCOUNT_KEY='{"type":"service_account","project_id":"..."}'
```

No **Vercel**: `Settings → Environment Variables → FIREBASE_SERVICE_ACCOUNT_KEY`

`lib/firebase-admin.ts` já está configurado para ler essa variável automaticamente.

Painel Administrativo

Rota `/admin`

Página client-side com duas abas: **Cadastrar** e **Roles**.

Cadastrar Lulu

- Formulário com campos: `fullName`, `name`, `date`, `month`, `city`, `authEmail`, `role`
- Usa `useCreateParticipant()` → chama `participants-admin.ts` → `setDoc()` no Firestore
- ID auto-incrementado a partir do maior ID existente

Gerenciar Roles

- Lista todos os participantes com dropdown de role (`admin`, `lulu`, `visitante`)

- Ao mudar para `admin`, chama `/api/admin/set-claim` para setar Custom Claim
- Permite excluir participantes via `useDeleteParticipant()`

Serviço `participants-admin.ts`

Funções protegidas por `assertAdmin()`:

- `createParticipant(data)` → cria doc com ID sequencial
- `deleteParticipant(participantId)` → exclui doc
- `updateParticipantRole(participantId, role)` → atualiza campo `role`
- `fetchAllParticipantsAdmin()` → lista todos ordenados por nome

Self-Edit (Meu Perfil)

Rota `/meu-perfil`

Página onde lulus editam seu próprio cadastro. Requer `role === 'lulu'` ou `isAdmin`.

Componentes:

1. **PersonForm** (mode: `self-edit`) → campos básicos (nome, telefone, instagram, PIX)
2. **GiftProfileForm** → perfil de presentes com campos adicionais
3. **NotificationOptIn** → ativar/desativar push notifications

GiftProfileForm

Formulário com `useFieldArray` para lista de desejos dinâmica:

Campo	Tipo	Descrição
`wishList`	`WishListItem[]`	Lista de desejos (item, url, preço)
`shirtSize`	`ShirtSize`	PP, P, M, G, GG, XG
`shoeSize`	`string`	Número do calçado
`favoriteColor`	`string`	Cor favorita
`allergies`	`string`	Alergias e restrições
`address`	`Address`	Endereço para entrega
`hobbies`	`string`	Hobbies e interesses
`favoriteStore`	`string`	Loja/marca favorita
`giftNotes`	`string`	Observações para presentes

Validação via `giftProfileSchema` (Zod) em `validation.ts`.

Push Notifications (FCM)

Arquitetura

```
┌──────────────┐     ┌────────────────┐     ┌────────────────┐
│  Client      │────▶│  Service       │────▶│  Firestore     │
│  Opt-in      │     │  Worker (SW)   │     │  fcmTokens[]   │
└──────────────┘     └────────────────┘     └────────────────┘
       │                  ▲
       │                  │ push event
       ▼                  │
┌──────────────┐     ┌────────────────┐
│  FCM API     │────▶│  Firebase      │
│  (Server)    │     │  Messaging     │
└──────────────┘     └────────────────┘
```

Serviço Client (`services/fcm.ts`)

Função	Descrição
`getMessagingInstance()`	Retorna instância FCM (null se não suportado)
`requestNotificationPermission(participantId)`	Solicita permissão + salva token
`removeNotificationToken(participantId, token)`	Remove token do Firestore
`setupForegroundMessages()`	Exibe toast com `onMessage()` em foreground
`hasNotificationPermission()`	Verifica se `Notification.permission === 'granted'`
`getStoredFcmToken(participantId)`	Lê tokens FCM do documento

Service Worker (`app/sw.ts`)

Handlers adicionados ao Serwist SW:

- **`push`**: Mostra `showNotification()` com título, body e ícone
- **`notificationclick`**: Abre/foca a URL do `data.url` da notificação

API Routes

`POST /api/notifications/send`

Envia push para tokens FCM especificados.

- **Auth**: Bearer token de admin OU `CRON_SECRET`
- **Body**: `{ title, body, tokens, link? }`
- **Limpeza**: Remove tokens inválidos automaticamente via batch update

`GET /api/cron/birthday-notifications`

Cron job para notificações de aniversário.

- **Auth**: `Authorization: Bearer <CRON_SECRET>`
- **Lógica**: Verifica aniversários de hoje e daqui a 5 dias
- **Batch**: Envia em lotes de 500 tokens

Foreground Handler (`fcm-foreground-handler.tsx`)

Importado dinamicamente no `layout.tsx` → chama `setupForegroundMessages()` no mount.

CSP (Content Security Policy)

`next.config.ts` atualizado com os domínios FCM:

```
connect-src: https://fcm.googleapis.com https://push.services.mozilla.com
```

Variáveis de Ambiente (FCM)

Variável	Descrição
`NEXT_PUBLIC_FIREBASE_VAPID_KEY`	VAPID key para Web Push
`CRON_SECRET`	Secret para autenticar cron jobs

React Query Strategy

Configuração Global

```
// providers/react-query-provider.tsx
'use client';

import { QueryClient, QueryClientProvider } from '@tanstack/react-query';
import { ReactQueryDevtools } from '@tanstack/react-query-devtools';

const queryClient = new QueryClient({
  defaultOptions: {
    queries: {
      staleTime: Infinity,
      refetchOnWindowFocus: false,
      retry: 1,
    },
  },
});

export const ReactQueryProvider = ({ children }: { children: ReactNode }) => {
  return (
    <QueryClientProvider client={queryClient}>
      {children}
      <ReactQueryDevtools initialIsOpen={false} />
    </QueryClientProvider>
```

```
  );
};
```

Query Hooks Pattern

```
// services/queries/fetchParticipants.ts
import { useQuery } from '@tanstack/react-query';
import { collection, getDocs } from 'firebase/firestore';
import { db } from '../firebase';

export const useGetAllParticipants = () => {
  return useQuery({
    queryKey: ['participants'],
    queryFn: async () => {
      const snapshot = await getDocs(collection(db, 'participants'));
      return snapshot.docs.map((doc) => ({ id: doc.id, ...doc.data() }));
    },
  });
};
```

Mutation Hooks Pattern

```
// services/queries/updateParticipant.ts
import { useMutation, useQueryClient } from '@tanstack/react-query';
import { doc, updateDoc } from 'firebase/firestore';
import { db } from '../firebase';

export const useUpdateParticipantData = () => {
  const queryClient = useQueryClient();

  return useMutation({
    mutationFn: async ({ id, data }: { id: string; data: Partial<Person> }) => {
      const docRef = doc(db, 'participants', id);
      await updateDoc(docRef, data);
    },
    onSuccess: () => {
      queryClient.invalidateQueries({ queryKey: ['participants'] });
    },
  });
};
```

Tema e Estilização

Tailwind Configuration

```
// tailwind.config.ts
export default {
  darkMode: ['class'],
  content: ['./app/**/*.{ts,tsx}', './components/**/*.{ts,tsx}'],
  theme: {
    extend: {
      colors: {
        primary: 'hsl(var(--primary))',
        secondary: 'hsl(var(--secondary))',
        accent: 'hsl(var(--accent))',
        // ... custom colors
      },
      boxShadow: {
        'lulu-sm': '0 2px 8px rgba(255, 105, 180, 0.3)',
        'lulu-md': '0 4px 16px rgba(255, 105, 180, 0.4)',
        'lulu-lg': '0 8px 32px rgba(255, 105, 180, 0.5)',
      },
      keyframes: {
        'lulu-bounce': {
          '0%, 100%': { transform: 'translateY(0)' },
          '50%': { transform: 'translateY(-10px)' },
        },
      },
    },
  },
};
```

CSS Variables (globals.css)

```
@layer base {
  :root {
    --background: 0 0% 100%;
    --foreground: 0 0% 3.9%;
    --primary: 0 72.2% 50.6%;
    --primary-foreground: 0 0% 98%;
    /* ... */
  }

  .dark {
    --background: 0 0% 3.9%;
    --foreground: 0 0% 98%;
    /* ... */
  }
}
```

Theme Provider Usage

```
import { ThemeProvider } from 'next-themes';

<ThemeProvider
  attribute="class"
  defaultTheme="system"
  enableSystem
  disableTransitionOnChange
>
  {children}
</ThemeProvider>
```

Performance Otimizations

1. **Code Splitting**: Automático via Next.js App Router
2. **Memoization**: `React.memo()` em componentes pequenos
3. **React Query Cache**: Evita refetchs desnecessários
4. **Dynamic Imports**: Para componentes pesados
5. **Image Optimization**: Next.js `<Image>` component
6. **Turbopack**: Build mais rápido em desenvolvimento

Acessibilidade

Práticas Implementadas

- ☑ Semantic HTML (`<nav>`, `<main>`, `<aside>`)
- ☑ ARIA labels onde necessário
- ☑ Foco visível customizado
- ☑ Navegação por teclado completa
- ☑ Skip to content link
- ☑ Color contrast (WCAG AA)
- ☑ Screen reader text (`sr-only`)
- ☑ Form labels explícitas

Exemplo

```
<button
  aria-label="Curtir foto"
  aria-pressed={isLiked}
  onClick={handleLike}
  className="focus:outline-none focus:ring-2 focus:ring-primary"
>
  <Heart className={isLiked ? 'fill-red-500' : ''} />
</button>
```

← Voltar ao README